# Automated Irregular Shaped Stone Stacking via Pose Optimization and GPU-accelerated Monte Carlo Tree Search

Author Names Omitted for Anonymous Review. Paper-ID [add your ID here]

***Abstract*—Stacking irregular objects into prescribed target shapes without container walls requires mutually supporting arrangements despite uneven contact surfaces. The problem couples combinatorial object selection and ordering with continuous pose optimization, as each placement redistributes loads and changes support geometry and space available for subsequent objects. We propose a framework with two complementary components: structure-aware bilevel pose optimization and parallel Monte Carlo Tree Search (MCTS). The optimizer combines target-shape objectives with contact-force optimization to account for force and torque equilibrium across the structure under friction constraints, using bodywise ADMM updates for GPU parallelization. The receding-horizon MCTS planner evaluates batches of candidate placements in parallel through pose refinement and physical simulation, providing lookahead over object choices and placement poses. We hypothesize that combining structural stability evaluation with lookahead improves target coverage while maintaining stability under a fixed planning budget.**

## I. Introduction

Stacking is a fundamental problem with applications in bin packing, industrial warehouse storage [20, 9], and stone-wall construction [12]. In freestanding stacking, the resulting structure must approximate a prescribed target shape while maintaining force and torque equilibrium under contact forces and gravity, without support from an enclosing container. For $n$ irregular objects, this geometric goal can be expressed as minimizing the unfilled volume of a target shape $T \subset \mathbb{R}^3$, subject to physical feasibility:

$$\min_{q_1,\cdots,q_n \in \mathrm{SE}(3)} \mathbb{V}\left(T \setminus \bigcup_{i=1}^{n} O_i(q_i)\right) \quad \text{s.t.} \quad c_{\text{phys}}(O_1(q_1),\cdots,O_n(q_n)) = 0, \tag{1}$$

where $O_i(q_i) \subset \mathbb{R}^3$ is the region occupied by object $i$ at pose $q_i \in \mathrm{SE}(3)$, $\mathbb{V}(\cdot)$ denotes volume, and $c_{\text{phys}} = 0$ represents stability and non-interpenetration constraints. Achieving these requirements simultaneously is challenging because changing one object pose alters both the occupied space and the contact geometry through which loads are transmitted across the structure. Moreover, the choice and pose of lower objects constrain the remaining feasible subsequent placements, so even a currently stable arrangement may limit further stacking toward the target shape, especially when the available object inventory is limited. Planning its construction therefore involves combinatorial choices of objects and placement order, together with iterative numerical evaluation of contact interactions [7], making planning computationally expensive.



Fig. 1: Preliminary stacking results. Left: simulated structures for different target shapes. Right: a physical stone structure from an outdoor experiment.

Robotic bin-packing methods consider placement stability within containers [20, 9], where enclosing walls can provide lateral support unavailable in freestanding construction. More directly related, Johns et al. [12, 13] construct freestanding stone walls using contact-aware registration and simulation, while Liu and Napp [16] combine heuristic filtering with simulation-based assessments of structural strength to select placements. These stone-stacking methods do not use equilibrium across the entire structure to guide continuous pose refinement. Furthermore, they select placements using the current structure without explicitly comparing subsequent placement sequences, although masonry-inspired rules encourage suitable support for later objects. This leaves two challenges: refining poses according to contact-force redistribution throughout the structure and evaluating how current choices affect future feasible placements.

We address these challenges through pose optimization incorporating structural stability and GPU-accelerated Monte Carlo Tree Search (MCTS) for sequence planning. Unlike prior stacking methods, we formulate placement refinement as a constrained bilevel problem and compute pose gradients of the objective and constraints using differentiable support functions (DSFs) for shape representation and collision detection [14, 2]. Bilevel formulations also appear in contact-aware manipulation planning and grasp generation [15, 21, 5]. Our lower-level problem accounts for force and torque equilibrium of all stones, including the candidate, with shared contact forces transmitting loads through the stack to the ground.

Evaluating how current placements affect subsequent stacking through MCTS requires repeated pose refinement and

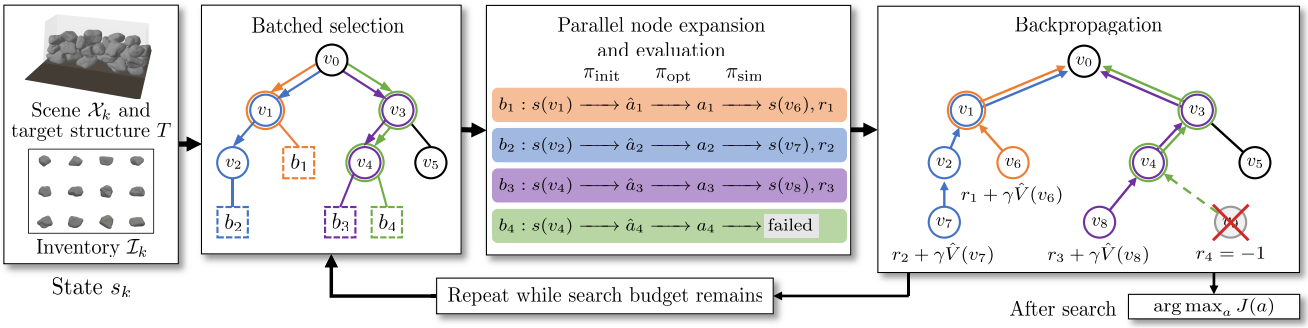



Fig. 2: Batched MCTS sequence planning. Colored requests are evaluated in parallel. Returns combine rewards and discounted leaf estimates. The dashed branch backpropagates its failure penalty once ($r_4 = -1$, $\hat{V}(v_9) = 0$) and is never selected again. After search, $J(a)$ ranks eligible root actions. See Sec. V-D for $\hat{V}(v)$ and $J(a)$.

structural simulation, making lookahead computationally demanding. To make lookahead search computationally practical for freestanding stacking, we introduce bodywise contact-force copies and an alternating direction method of multipliers (ADMM) decomposition for parallel force optimization within each candidate structure, while batching independent candidate placements on the GPU. Together with GPU-accelerated subADMM simulation [11], this enables parallel candidate evaluation throughout the search tree (Figure 2). Heuristic leaf evaluation replaces costly rollouts, while task-specific rewards guide the search toward target coverage and structural stability.

Our main contributions are as follows:

- We formulate a single pose refinement for freestanding stacking as a constrained bilevel placement problem, using differentiable contact features and geometric derivatives to guide pose refinement with target geometry and whole-structure equilibrium residuals.
- We introduce a bodywise ADMM decomposition of the contact-force problem, enabling GPU-parallel solves within each candidate structure and across candidate placements.
- We develop a GPU-accelerated MCTS planner that significantly accelerates lookahead search over object choices and placement poses through parallel pose optimization and structural simulation.

## II. Related Works

### A. Robotic Packing and Stone Stacking

Wang and Hauser [20] check pile equilibrium in containers using linear programming with pyramidal friction cones, without using equilibrium residuals for continuous pose refinement.

Johns et al. [12, 13] construct freestanding walls using masonry-inspired placement rules and torque and intersection constrained registration (TICR). TICR combines candidate contact-force optimization, iterative closest point registration, and heuristic rotational corrections, followed by simulation for stability evaluation. Pose updates do not account for load redistribution across the structure.

Liu et al. [17] geometrically filter simulation-settled poses; Liu and Napp [16] add shake tests to select the candidate with the longest time to collapse above a threshold, without searching future placements. Testing all feasible poses without filtering increases runtime by 5–15× [16]. Our method uses whole-structure equilibrium residuals for pose refinement and MCTS for subsequent placement sequences.

### B. Contact-Aware Pose Optimization

Lee et al. [15] model manipulation with contact factor graphs, using differentiable contact features and the envelope theorem for configuration inference after conditional force optimization, assuming a predefined task skeleton. GraspQP [21] defines a force-closure energy through a differentiable QP and uses KKT differentiation to guide grasp refinement. BODex [5] uses bilevel grasp optimization with pyramidal friction cones and batched ReLU-QP, an ADMM-based GPU solver for contact forces.

### C. Simulation-Based Sequential Manipulation Planning

PMBS [8] combines parallel MCTS with GPU rigid-body simulation for object retrieval from clutter. Our planner adopts batched evaluation and additionally parallelizes whole-structure contact-force optimization for pose refinement before structural simulation at each expansion.

## III. Preliminaries and Overview

### A. Shape Representation

We represent the target shape $T$ and each object $O_i$ as unions of smooth convex bodies modeled by differentiable support functions (DSFs) [14]. For a compact convex body $C \subset \mathbb{R}^3$, its support function is $h_C(x) = \max_{p \in C} x^\top p$ for $x \neq 0$. At pose $q = (R, t) \in \mathrm{SE}(3)$, we write

$$C(q) = \{Rp + t \mid p \in C\}, \qquad O_i(q_i) = \bigcup_j C_{ij}(q_i),$$

where $C_{ij}$ are the convex components in object $i$'s local frame. Similarly, $T = \bigcup_j T_j$, where $T_j$ are fixed convex components in the world frame. DSFs support differentiable signed-distance and contact-geometry computations between posed convex bodies [2], enabling gradient-based pose optimization.

### B. Markov Decision Process Formulation

Given an inventory of $n$ objects, we model stacking as a finite-horizon sequential decision process in which one object is placed at each step. Let $\mathcal{P}_k \subseteq \{1,\dots,n\}$ be the indices of the objects already placed, and let $i_0 \notin \{1,\dots,n\}$ index the fixed ground. The current scene is $\mathcal{X}_k = \{(i, q_i) \mid i \in \mathcal{P}_k\}$, and the remaining inventory is $\mathcal{I}_k = \{1,\dots,n\} \setminus \mathcal{P}_k$. The state, action, and deterministic transition function are

$$s_k = (\mathcal{X}_k, \mathcal{I}_k, T), \quad a_k = (i_{k+1}, q_{k+1}), \quad s_{k+1} = F(s_k, a_k),$$

where $i_{k+1} \in \mathcal{I}_k$. The transition inserts the proposed pose into the scene, simulates the resulting structure, and returns the settled scene with $\mathcal{I}_{k+1} = \mathcal{I}_k \setminus \{i_{k+1}\}$. The reward $r(s_k, a_k)$ scores target-contained volume and physical quality; feasibility checks separately reject invalid transitions (Sec. V). The sequence planner therefore operates on the MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, F, r, \gamma)$.

### C. Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) incrementally constructs a search tree $\mathcal{T} = (\mathcal{V}, \mathcal{E})$ over the MDP. Each node $v \in \mathcal{V}$ represents a state $s(v)$, and each edge $(v, a) \in \mathcal{E}$ represents an action and its transition to a child node. Let $N(v)$ denote the number of completed evaluations and $Q(v)$ the empirical mean of discounted returns backpropagated to that node.

Each iteration selects a path by balancing value and exploration, expands and evaluates an action, and backpropagates its discounted return to update $Q$ and $N$.

In our planner (Fig. 2), expansion uses three modules: $\pi_{\text{init}}$ proposes an object and initial pose $\hat{a}_k = (i_{k+1}, \hat{q}_{k+1})$, $\pi_{\text{opt}}$ refines the pose, and $\pi_{\text{sim}}$ evaluates the resulting structure to determine feasibility, the settled successor, and reward. The resulting flow is

$$s_k \xrightarrow{\pi_{\text{init}}} \hat{a}_k \xrightarrow{\pi_{\text{opt}}} a_k \xrightarrow{\pi_{\text{sim}}} s_{k+1},\ r(s_k, a_k).$$

Sec. V describes how these modules are integrated into batched search.

## IV. Pose Optimization

The module $\pi_{\text{opt}}$ refines $\hat{q}_{k+1}$ from $\pi_{\text{init}}$ toward a stable, physically feasible placement $q_{k+1}$.

### A. Single Object Pose Optimization Problem

Given $s_k$, we initialize $q$ at $\hat{q}_{k+1}$ and solve the bilevel problem

$$\begin{aligned} \min_{q} \quad & \left[ f_{\text{geo}}(q) + \min_{\lambda} f_{\text{phys}}(q, \lambda) \right] \\ \text{s.t.} \quad & g_{\text{cone},j}(q, \lambda_j) \le 0, \quad j = 1, \cdots, n_c \end{aligned} \tag{2}$$

where $q \in \text{SE}(3)$ is the candidate object's pose and $\lambda = (\lambda_1, \cdots, \lambda_{n_c}) \in \mathbb{R}^{3n_c}$ collects all contact forces in the resulting structure, including inter-object and ground contacts. This formulation follows prior contact-aware optimization methods [15, 5].

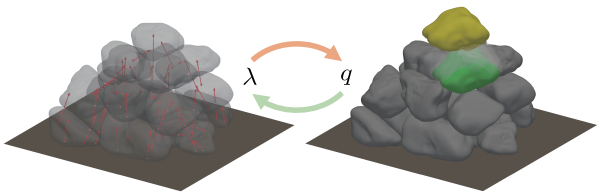



Fig. 3: Bilevel pose optimization. Left: optimizing contact forces $\lambda$ under friction constraints minimizes structural equilibrium residuals at fixed $q$. Right: refining $q \in \text{SE}(3)$ minimizes the geometric and optimized physical objectives.

The geometric objective $f_{\text{geo}}$ is designed as follows:

$$\begin{aligned} f_{\text{geo}}(q) = w_{\text{h}} f_{\text{h}}(q) + w_{\text{in}} f_{\text{in}}(q \mid T, O_{i_{k+1}}) \\ + w_{\text{pen}} f_{\text{pen}}(q \mid \mathcal{X}_k, O_{i_{k+1}}) \end{aligned}$$

with positive weights. For $q = (R, t)$, $f_{\text{h}}(q) = t_z$ is the world-frame height of the candidate's pose origin. Writing $O = O_{i_{k+1}}$, we penalize proximity to existing objects and the fixed ground as

$$f_{\text{pen}}(q) = \sum_{j \in \mathcal{P}_k \cup \{i_0\}} \left[ \epsilon_{\text{margin}} - \text{d}(O(q), O_j(q_j)) \right]_+^2,$$

where d is signed distance (positive for separation), $[x]_+ = \max(x, 0)$, and $\epsilon_{\text{margin}} \ge 0$ is the clearance margin. With $h_{O(q)}(u) = \max_j h_{C_{i_{k+1},j}(q)}(u)$, we define

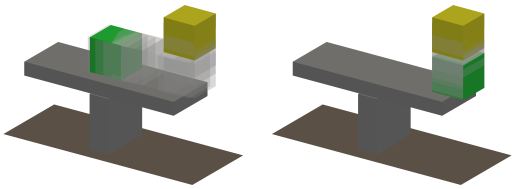
$$\begin{aligned} m_{\text{in}}(q) &= \max_j \min_{u \in \mathbb{S}^2} \left[ h_{T_j}(u) - h_{O(q)}(u) \right], \\ f_{\text{in}}(q) &= \left[ -m_{\text{in}}(q) \right]_+^2. \end{aligned}$$

This term penalizes violations of containment of the entire object within a single convex target component, a conservative condition for containment in $T$. Noting that the term is differentiable almost everywhere in $q$, we compute pose derivatives at regular inner solutions by implicit differentiation. Nonsmooth transitions have not caused practical difficulties in our implementation.

Let $\mathcal{P}_{k+1} := \mathcal{P}_k \cup \{i_{k+1}\}$. For distinct $l, m \in \mathcal{P}_{k+1} \cup \{i_0\}$, let $I_{l,m} \subset \{1,\dots,n_c\}$ index contacts between bodies $l$ and $m$. Each object's contact set $I_l := \bigcup_{m \in (\mathcal{P}_{k+1} \cup \{i_0\}) \setminus \{l\}} I_{l,m}$ includes ground contacts, but the physical objective excludes the ground's equilibrium residual:

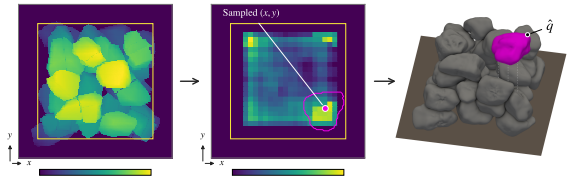
$$\begin{aligned} f_{\text{phys}}(q, \lambda) = w_{\text{eq}} \sum_{l \in \mathcal{P}_{k+1}} \| J_{c,I_l}^{\top}(q) \lambda_{I_l} + \lambda_{\text{ext},l} \|_W^2 \\ + w_{\text{cp}} \sum_{j \in I_{i_{k+1}}} \text{d}_j(q)^2 \|\lambda_j\|^2 + w_{\text{reg}} \sum_{j=1}^{n_c} \|\lambda_j\|^2 \end{aligned} \tag{3}$$

The three terms respectively penalize violations of static equilibrium, contact complementarity between the candidate and the current scene, and large contact forces through regularization. Figure 4 illustrates why equilibrium is evaluated across the structure rather than only for the candidate. The weight matrix $W \succ 0$ balances force and moment scales. Here, $\text{d}_j(q)$ is the signed distance between the bodies of contact candidate



(a) Structure-wide (b) Candidate-only

Fig. 4: Seesaw example: initial (yellow), optimized (green), and intermediate (gray) poses. Structure-wide equilibrium (a) moves the candidate toward the support; candidate-only equilibrium (b) leaves it near the unsupported end.




(a) Scene height (b) Proposal score (c) Sampled initial pose

Fig. 5: Initial pose sampling for a fixed object and orientation: (a) scene height, (b) proposal score and sampled planar location, and (c) initial pose $\hat{q}$ above the scene. Magenta links the sampled location and candidate object.

$j$; $J_{c,I_l}$, $\lambda_{\text{ext},l}$, and $\lambda_{I_l}$ are object $O_l$'s contact Jacobian, external wrench, and contact-force vector, respectively; and $w_{\text{eq}}, w_{\text{cp}}, w_{\text{reg}} > 0$ are objective weights.

Finally, each $g_{\text{cone},j}(\cdot)$ is the smoothed friction cone constraints for the $j$-th contact point defined by

$$g_{\text{cone},j}(q, \lambda_j) = \sqrt{\|J^t_{c,j}\lambda_j\|^2 + \epsilon} - \mu_j J^n_{c,j}\lambda_j \tag{4}$$

where $\epsilon > 0$ is a smoothing parameter, $\mu_j$ is the friction coefficient, and $J^t_{c,j}$ and $J^n_{c,j}$ are the tangential and normal components of the Jacobian of the contact constraint.

### B. Upper-Level Problem Optimization on SE(3) Manifold

Let $\lambda^*(q)$ be the solution of the lower-level problem and define $f_{\text{phys}}(q) := f_{\text{phys}}(q, \lambda^*(q))$. The envelope theorem [4] gives

$$\frac{df_{\text{phys}}(q)}{dq} = \frac{\partial f_{\text{phys}}(q, \lambda^*)}{\partial q} + \sum_{j=1}^{n_c} \nu_j^* \frac{\partial g_{\text{cone},j}(q, \lambda_j^*)}{\partial q}$$

where $\nu_j^*$ is the optimal dual variable for the $j$-th friction cone constraint. Using the above gradient, we optimize the placement pose $q$ on SE(3) manifold based on the Riemannian trust-region method [1] combined with BFGS Hessian approximation [10]. A homotopy schedule adjusts the geometric and physical weights between stages, holding them fixed within each stage and initializing from the preceding pose. It progressively reduces $w_{\text{reg}}$ to limit regularization-induced equilibrium residuals. Candidate evaluation uses the final pose and forces.

### C. Lower-Level Problem Optimization with ADMM

For fixed $q$, the lower-level problem is convex in $\lambda$. To apply bodywise ADMM [3], we introduce two local copies $\lambda_{j,1}$ and $\lambda_{j,2}$ per contact, one for each contacting body, including the ground. Collecting these copies into $\tilde{\lambda}$ makes each body's equilibrium residual depend only on its local variables. Consensus and friction constraints give the reformulation

$$\begin{aligned} \min_{\tilde{\lambda}, z} \quad & \tilde{f}_{\text{phys}}(q, \tilde{\lambda}) + \sum_{\substack{j=1,\dots,n_c \\ s=1,2}} \delta_{K_j}(z_{j,s}) \\ \text{s.t.} \quad & \lambda_{j,1} = \lambda_{j,2} = z_{j,1} = z_{j,2}, \qquad j = 1, \dots, n_c. \end{aligned}$$

where $\tilde{f}_{\text{phys}}$ is the physical objective in local variables, $z_{j,s}$ are auxiliary variables, and $\delta_{K_j}$ is the indicator of $K_j := \{\lambda \in \mathbb{R}^3 \mid g_{\text{cone},j}(q, \lambda) \le 0\}$. ADMM solves the body subproblems independently. An equivalent four-dimensional second-order cone (SOC) lifting enables closed-form cone projections for the friction updates, while consensus updates reduce to averaging.

The pose optimization module is implemented using CUDA to leverage GPU parallelization for efficient computation of the gradients and the ADMM updates.

## V. Sequence Planning via Monte Carlo Tree Search

### A. Initial Pose Sampling

The sampler $\pi_{\text{init}}$ uses volume-biased object sampling with a finite orientation set per object. It favors lower placements and seeds empty scenes near feasible target corners. For object $i$, orientation $R$, and feasible planar location $\mathbf{x}$, the score (Fig. 5) is

$$\sigma(\mathbf{x} \mid i, R) = w_{\text{s}}\sigma_{\text{s}} + w_{\text{f}}\sigma_{\text{f}} + w_{\text{b}}\sigma_{\text{b}} - w_{\text{h}}\sigma_{\text{h}} - w_{\text{d}}\sigma_{\text{d}},$$

with positive weights. The first four terms measure the fraction of bottom samples near predicted contact, unfilled layer area beneath the footprint, proximity to the feasible placement boundary, and normalized mean contact height, respectively. A bounded penalty $\sigma_{\text{d}}$ on the planar footprint gap to the preceding object (zero for the first placement) encourages nearby placements, helping MCTS assess their interactions without excluding distant proposals. Inspired by Menezes et al. [18], the floor and boundary terms encourage filling empty layer regions and building inward from the boundary.

We combine elite selection with sampling without replacement weighted by $\exp(\sigma/\tau)$, $\tau > 0$. Proposals start above the scene with clearance and undergo $\pi_{\text{opt}}$ refinement, including XY/yaw mutations at the root. Heuristic filters limit planar displacement from each initial pose after optimization and settling, and screen out candidates whose bottoms lie too far above the current construction level or whose equilibrium residual is too large (defined below).

### B. Simulation Module

The module $\pi_{\text{sim}}$ uses the same simulation procedure at every search depth. It settles the new object for a fixed duration

with existing objects fixed, then resets velocities and simulates the entire structure's response to the added load. Structural simulation requires a minimum duration and sustained low mass-normalized kinetic energy. Excessive accumulated motion, invalid contacts, or failure to settle within the time limit rejects the placement.

The accepted settled scene defines the successor $s' = F(s,a)$. Each disturbance trial starts from rest in this scene and applies a horizontal force at the new object's center of mass:

$$\mathbf{F}_j = \alpha m_{i_{k+1}} g\big(\cos\theta_j,\ \sin\theta_j,\ 0\big)^\top,$$

where $\theta_j = 2\pi j/n_d$ for $j = 0,\dots,n_d-1$, $\alpha > 0$ sets the force relative to object weight, and $g$ is gravitational acceleration. The force is removed after $\Delta t_f$ to observe recovery; all placed objects remain dynamic throughout. Every trial must recover within prescribed tolerances without excessive pose excursion. These trials assess structural robustness without changing $s'$. GPU-accelerated subADMM [11] enables parallel candidate simulation, complementing the parallel pose optimizer for batched MCTS evaluation.

### C. Reward Design

For the newly placed object $i = i_{k+1}$, let $q_i'$ be its pose in the settled successor $s'$ returned by $\pi_{\text{sim}}$ (Sec. V-B). We define the target-volume and object-volume ratios

$$\rho_T = \frac{\mathbb{V}\big(O_i(q_i') \cap T\big)}{\mathbb{V}(T)}, \quad \rho_O = \frac{\mathbb{V}\big(O_i(q_i') \cap T\big)}{\mathbb{V}(O_i)}.$$

Here $\mathbb{V}(\cdot)$ denotes volume. The $\rho_T$ term encourages target filling, while $\rho_O$ discourages placements extending excessively outside the target. Intersection volumes are estimated on a voxel grid. Failed transitions receive $r(s,a) = -1$. For feasible transitions, with fixed normalization scales absorbed into positive weights, the reward is

$$\begin{aligned} r(s,a) = w_T\rho_T + w_O[\rho_O - \tfrac{1}{2}]_+ - w_E e(a) \\ + w_H\big(\phi_H(s) - \phi_H(s')\big) - w_D D, \end{aligned}$$

where $[x]_+ = \max(x,0)$, so the containment bonus is active above $50\%$. Coverage lost through motion of earlier objects is not subtracted. The height term rewards the decrease in surface-height variance $\phi_H(s) = \langle (H_s - \langle H_s\rangle_\Omega)^2\rangle_\Omega$ between parent and settled successor. Here $H_s$ is height over the target footprint $\Omega$, zero at empty cells, and $\langle\cdot\rangle_\Omega$ denotes the area-weighted mean.

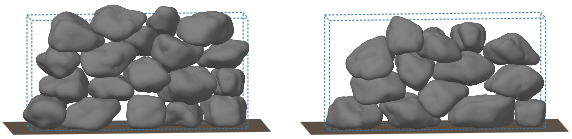
The equilibrium score uses residual forces $\mathbf{f}_l$ and torques $\boldsymbol{\tau}_l$ returned by pose optimization:

$$e(a) = \max_l \left( \frac{\|\mathbf{f}_l\|}{m_l g} + \frac{\|\boldsymbol{\tau}_l\|}{m_l g \mathbb{V}(O_l)^{1/3}} \right).$$

Here $l$ indexes objects in the resulting structure and $g$ is gravitational acceleration. Candidates with $e(a) \geq \epsilon_{\text{eq}}$ are rejected after pose optimization, before simulation, so the equilibrium term ranks only admitted placements. Let $q_l^{(j)}(t)$ denote object $l$'s pose during force trial $j$ (Sec. V-B), starting from its nominal successor pose $q_l'$. Disturbance sensitivity is

$$D = \min\left\{ D_{\max}, \max_{j,l,t} d_{\text{SE(3)}}\big(q_l^{(j)}(t), q_l'\big) \right\},$$

where $d_{\text{SE(3)}}$ sums Euclidean translation distance and angular distance weighted by $\lambda_\theta > 0$. The maximum spans force trials, all placed objects, and simulation time including recovery; $D_{\max}$ caps the penalty. Physical rejection criteria apply independently of the reward.



(a) MCTS (b) Greedy

Fig. 6: Wall construction using the same inventory of over 40 stones. MCTS places 19 stones ($44.0\%$ target filling); greedy stops at 14 ($36.1\%$), finding no further feasible placement. Blue dashed outlines show the target.

### D. Monte Carlo Tree Search

We integrate these modules in batched, receding-horizon MCTS (Fig. 2), following the parallel evaluation approach of [8]. Figure 6 shows a recorded comparison with greedy placement.

**Batched Selection.** For an optimized action $a = (i,q)$, let $\rho_O^{\text{pre}}$ denote the object-volume ratio evaluated at $q$, before simulation. Using the reward weights, the logit $\ell(a) = w_O\rho_O^{\text{pre}} - w_E e(a)$ defines $P(v_p,a)$ through a softmax over selectable children. Writing $N_v = N(v)$ and similarly for $Q_v$ and $\hat{N}_v$, an evaluated child $v$ of $v_p$ is selected using PUCT [19]:

$$\text{PUCT}(v) = \frac{N_v Q_v}{N_v + \hat{N}_v} + cP(v_p,a)\frac{\sqrt{N_{v_p} + \hat{N}_{v_p}}}{1 + N_v + \hat{N}_v}, \qquad (5)$$

where $c > 0$ controls exploration, $\hat{N}$ counts temporary reservations, and the first term is zero when its denominator is zero. At non-root nodes, progressive widening [6] increases the number of candidate actions with completed visits. Temporary reservations prevent duplicate evaluations within a batch without increasing the widening allowance.

**Parallel Expansion and Evaluation.** After batch selection, fresh proposals pass through $\pi_{\text{init}} \to \pi_{\text{opt}} \to \pi_{\text{sim}}$. Independent candidates are processed in parallel; selection never descends through an action without an evaluated successor. Failed candidates are excluded from further selection.

**Leaf Value Estimation and Backpropagation.** We replace rollouts with a heuristic estimate of future rewards:

$$\hat{V}(v) = \begin{cases} 0, & \text{if failed or terminal,} \\ \hat{r}\sum_{j=0}^{d_v-1}\gamma^j, & \text{otherwise,} \end{cases}$$

where $\hat{r}$ is a prescribed per-step reward estimate and $d_v$ the bootstrap horizon, capped by remaining depth and inventory (zero at the depth limit). Backup starts from $r_v + \gamma \hat{V}(v)$, where $r_v$ is the incoming action's reward, and recursively adds ancestor rewards with discount $\gamma$, updating $Q$, $N$, and clearing temporary reservations.

**Action Selection and Tree Reuse.** For each feasible root action $a$, let $G(a)$ be the maximum discounted observed return among its deepest evaluated feasible paths within the search horizon, including $a$'s reward and excluding bootstrap values. Root-only paths are eligible. We select the action maximizing

$$J(a) = (1 - \beta)Q(v_a) + \beta G(a), \qquad \beta \in [0, 1].$$

The path-return term counterbalances failures accumulated during exploration, which can depress the mean even when a promising feasible path exists. If no root action is feasible, we return no placement.

After execution, we advance the root to the observed successor. Evaluated descendants are retained when this state and the planning context match those used to construct the subtree; otherwise, the affected subtree and its statistics are discarded.

## Acknowledgments

## References


[1] P. A. Absil, C. G. Baker, and K. A. Gallivan. Trust-region methods on riemannian manifolds. *Foundations of Computational Mathematics*, 7:303–330, 2007.

[2] S. An, S. Lee, J. Lee, S. Park, and D. Lee. Collision detection between smooth convex bodies via riemannian optimization framework. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12464–12471. IEEE, 2024.

[3] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1):1–122, 2011. doi: 10.1561/2200000016.

[4] M. Carter. *Foundations of mathematical economics*. MIT press, 2001.

[5] J. Chen, Y. Ke, and H. Wang. BODex: Scalable and efficient robotic dexterous grasp synthesis using bilevel optimization. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1–8. IEEE, 2025. doi: 10.1109/ICRA55743.2025.11127930.

[6] A. Couëtoux, J.-B. Hoock, N. Sokolovska, O. Teytaud, and N. Bonnard. Continuous upper confidence trees. In *Learning and Intelligent Optimization*, volume 6683 of *Lecture Notes in Computer Science*, pages 433–445. Springer, 2011. doi: 10.1007/978-3-642-25566-3_32.

[7] F. Furrer, M. Wermelinger, H. Yoshida, F. Gramazio, M. Kohler, R. Siegwart, and M. Hutter. Autonomous robotic stone stacking with online next best object target pose planning. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2350–2356, 2017. doi: 10.1109/ICRA.2017.7989272.

[8] B. Huang, A. Boularias, and J. Yu. Parallel monte carlo tree search with batched rigid-body simulations for speeding up long-horizon episodic robot planning. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022.

[9] S. Huang, Z. Wang, J. Zhou, and J. Lu. Planning irregular object packing via hierarchical reinforcement learning. *IEEE Robotics and Automation Letters*, 8(1): 81–88, 2023. doi: 10.1109/LRA.2022.3222996.

[10] W. Huang, P.-A. Absil, and K. A. Gallivan. A riemannian BFGS method without differentiated retraction for nonconvex optimization problems. *SIAM Journal on Optimization*, 28(1):470–495, 2018. doi: 10.1137/17M1127582.

[11] H. Ji, H. Kim, J. Lee, S. Lee, S. An, J. Heo, Y. Lee, Y. Lee, and D. Lee. Gpu-accelerated subsystem-based admm for large-scale interactive simulation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 13868–13874. IEEE, 2025.

[12] R. L. Johns, M. Wermelinger, R. Mascaro, D. Jud, F. Gramazio, M. Kohler, M. Chli, and M. Hutter. Autonomous dry stone. *Construction Robotics*, 4(3–4):127–140, 2020. doi: 10.1007/s41693-020-00037-6.

[13] R. L. Johns, M. Wermelinger, R. Mascaro, D. Jud, I. Hurkxkens, L. Vasey, M. Chli, F. Gramazio, M. Kohler, and M. Hutter. A framework for robotic excavation and dry stone construction using on-site materials. *Science Robotics*, 8(84):eabp9758, 2023. doi: 10.1126/scirobotics.abp9758.

[14] J. Lee, M. Lee, and D. Lee. Uncertain pose estimation during contact tasks using differentiable contact features. *RSS 2023-Robotics: Science and Systems*, 2023.

[15] J. Lee, S. Park, M. Lee, and D. Lee. Efficient gradient-based inference for manipulation planning in contact factor graphs. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 15401–15407. IEEE, 2025. doi: 10.1109/ICRA55743.2025.11127769.

[16] Y. Liu and N. Napp. Stability-based sequence planning for robotic dry-stacking of natural stones. *IEEE Robotics and Automation Letters*, 8(9):5894–5901, 2023. doi: 10.1109/LRA.2023.3301275.

[17] Y. Liu, J. Choi, and N. Napp. Planning for robotic dry stacking with irregular stones. In G. Ishigami and K. Yoshida, editors, *Field and Service Robotics*, volume 16 of *Springer Proceedings in Advanced Robotics*, pages 321–335. Springer, 2021. doi: 10.1007/978-981-15-9460-1_23.

[18] A. Menezes, P. Vicente, A. Bernardino, and R. Ventura. From rocks to walls: A model-free reinforcement learning approach to dry stacking with irregular rocks. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 2057–2065, 2021. doi: 10.1109/CVPRW53098.2021.00234.

[19] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis. Mastering the game of go without human knowledge. *Nature*, 550:354–359, 2017.

[20] F. Wang and K. Hauser. Dense robotic packing of irregular and novel 3D objects. *IEEE Transactions on Robotics*, 38(2):1160–1173, 2022. doi: 10.1109/TRO.2021.3097261.

[21] R. Zurbrügg, A. Cramariuc, and M. Hutter. GraspQP: Differentiable optimization of force closure for diverse and robust dexterous grasping. In *Conference on Robot Learning (CoRL)*, pages 2583–2602. PMLR, 2025.